Mbarara University of Science and Technology
SWE2106 Internet Technologies and Web Design

Digital Class Attendance System

Group B Members

No.	Name	Registration No.
1	Wokwaba Charles	2025/BSE/187/PS
2	Kanyesigye Brian	2025/BSE/080/PS
3	Kwesiga Lewis	2025/BSE/097/PS
4	Akatuwijuka Annibow	2025/BSE/030/PS
5	Natweta Cyril	2025/BSE/132/PS
6	Wawangula Generous	2025/BSE/184/PS
7	Mugendawala Mark Paul	2025/BSE/108/PS
8	Katto Andrew	2025/BSE/085/PS
9	Nabaweesi Patricia Mirembe	2025/BSE/120/PS
10	Matovu Edwin Levi	2025/BSE/105/PS
11	Amen Joel Osuta	2025/BSE/152/PS

1. Problem Statement

The paper-based attendance process has problems that affect lecturers, students and the registrar's of fice:

- **It eats lecture time.** Passing a sheet through a large hall takes several minutes, and the lecturer l oses teaching time.
- **It is easy to cheat.** A student can sign for a friend who is not in class, and nobody can tell.
- **Records get lost or damaged.** Sheets are misplaced, torn, or end up unreadable, and once that h appens the data is gone.
- **Compiling the data is slow and error-prone.** Counting signatures by hand at the end of the seme ster leads to mistakes, and those mistakes can affect a student's eligibility.
- **There is no overall view.** Heads of department and the registrar cannot easily see attendance ac ross courses or faculties without collecting sheets from each lecturer.

2. Proposed Solution

We propose a web-based attendance system built with JavaScript: a Node.js and Express backend that exposes a REST API, a live database, and a simple frontend that lecturers and students can open on a phone or laptop. The basic flow is:

1. The lecturer logs in, selects a course unit, and starts an attendance session. The system generate s a short code (for example K7M2QX) that is only valid for a set time, such as 10 minutes.
2. The lecturer shows the code on the projector or board.
3. Each student logs in with their account and submits the code from their own phone.
4. The server checks that the student is registered for that course, that the session is still open, and

that the student has not already signed in. If everything is fine the attendance is saved with the exact time.

5. The lecturer closes the session, reviews the list, and can correct genuine cases (for example a student whose phone died).
6. Students can view their own attendance percentage per course, and authorized staff can pull reports at course or department level.

Because the code is short-lived and only shown in the lecture hall, it is much harder to sign in for someone who is not present than it is on a paper sheet. Every submission is also tied to a logged-in account and a timestamp, so there is a record of who did what.

3. Users

User	Who they are	What they do in the system
Student	Registered undergraduate or postgraduate students	Log in, submit the session code, view their own attendance history and percentage per course
Lecturer	Staff assigned to one or more course units	Start and close sessions, view live attendance, correct entries, download attendance for their own courses
Admin (Registrar's office / department)	Registrar staff and heads of department	Manage users, courses and enrollments, view reports across courses, set the minimum attendance threshold

4. Objectives

Main objective: to replace paper attendance sheets with a reliable digital system that saves lecture time and gives accurate attendance data to students, lecturers and the registrar.

Specific objectives:

- Let a student record attendance in under 30 seconds once the session is open.
- Prevent duplicate and out-of-time attendance entries through server-side validation.
- Store all records in a database so they cannot be lost like paper sheets.
- Give students access to their own attendance percentage at any time.
- Generate course-level attendance reports and flag students below the required minimum.
- Build a documented and tested API that other registrar systems (for example results or exam eligibility) could connect to later.

5. Scope

In scope (what we will build for this activity)

- User registration and login with three roles: student, lecturer, admin.
- Course units and student enrollment records.
- Creating, closing and listing attendance sessions with a time-limited code.
- Student attendance submission with validation.
- Attendance viewing for students (own record) and lecturers (their courses).
- Basic attendance summary and below-threshold report.
- A simple frontend connected to the live database, and a Postman collection for the API.

Out of scope (future work): biometric attendance, integration with the university's existing student system, native mobile apps, SMS or email notifications, and timetable import.

6. System Design

Architecture. The frontend (HTML, CSS and JavaScript) sends requests to the Express API. The API handles authentication, validation and business rules, and reads from and writes to the database. Keeping the rules on the server means the frontend cannot be used to bypass them.

Data. Five collections: users (name, email, hashed password, role), courses, enrollments, sessions (code, start and expiry time, status) and attendance (session, student, timestamp, status).

Planned API endpoints.

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Create an account	Admin
POST	/api/auth/login	Log in and receive a token	Everyone
POST	/api/courses/:id/enroll	Enroll a student in a course	Admin
POST	/api/sessions	Start an attendance session and generate a code	Lecturer
POST	/api/attendance	Submit the session code to mark attendance	Student
GET	/api/attendance/me	View own attendance history	Student
GET	/api/courses/:id/attendance	View attendance list for a course	Lecturer, Admin
GET	/api/reports/low-attendance	Students below the minimum threshold	Lecturer, Admin

Validation and security rules.

- Passwords are hashed with bcrypt and never stored or returned in plain text.
- Logins return a JSON Web Token, and every protected route checks the token and the user's role.
- A student can only submit attendance if they are enrolled in the course, the session is open and has not expired, and they have not already been marked for it.
- All input is checked (required fields, email format, code format) and bad requests return a clear message with the right status code: 400 for invalid input, 401 for not logged in, 403 for wrong role, 404 for not found, 409 for duplicates.

Tools. Node.js and Express for the backend, MongoDB Atlas as the live cloud database, jsonwebtoken, bcrypt and express-validator for security and validation, plain HTML, CSS and JavaScript for the frontend, Postman for API testing, draw.io for diagrams, and VS Code with Git and GitHub for development.

8. Testing Plan (Postman)

We will build a Postman collection, grouped by role, with sample data and saved environment variables for the base URL and tokens. For each key request we will test both the successful case and the failure cases:

Request	Valid case (expected)	Invalid cases (expected)
Login	200 and a token	Wrong password: 401. Missing email: 400
Start session	201 and a generated code	Student token: 403. Course not found: 404
Submit attendance	201 and attendance recorded	Wrong code: 400. Expired session: 400. Not enrolled: 403. Already marked: 409
View my attendance	200 and list with percentage	No token: 401
Low-attendance report	200 and list of students	Student token: 403

9. Conclusion

Attendance is a small daily task, but it adds up to a large amount of lost lecture time, paperwork and disputed records. A simple, well-validated web system can remove most of that effort, and building it as a documented API means the registrar's office could later connect it to examination eligibility or other services. We believe this project is realistic to deliver within the time we have, useful to the university, and a good way for our team to apply what we have learned about Express, REST design, databases and teamwork.